

ANTSDR E310 SDR 플랫폼으로의 PYNQ 3.1 프레임워크 포팅: 종합 하드웨어 분석 및 구현 가이드

섹션 1: 비교 아키텍처 분석: ANTSDR E310 대 PYNQ-Z2

PYNQ 프레임워크를 새로운 하드웨어에 포팅하는 작업의 성공 여부는 대상 보드와 기존 보드 간의 아키텍처 차이를 엄밀하게 분석하는 것에서 시작된다. 본 섹션에서는 소프트웨어 정의 라디오(SDR) 플랫폼인 ANTSDR E310과 범용 학술 및 프로토타이핑 보드인 PYNQ-Z2의 하드웨어적 차이점을 심층적으로 비교 분석한다. 두 보드는 동일한 SoC(System-on-Chip)를 공유하지만, 그 설계 철학의 근본적인 차이는 메모리, RF 프런트엔드, 주변 장치 인터페이스 등 핵심 구성 요소의 선택에서 명백하게 드러난다. 이러한 차이는 디바이스 트리 정의부터 커널 드라이버 요구사항에 이르기까지 포팅 전략 전반을 결정하는 핵심 변수로 작용한다.

1.1. 핵심 프로세싱, 메모리 및 스토리지 서브시스템

두 보드의 아키텍처적 공통점과 차이점은 시스템의 기본 성능과 잠재적 응용 분야를 규정한다.

- **SoC:** 두 보드 모두 Xilinx Zynq-7000 SoC, 구체적으로 **XC7Z020-1CLG400C** 모델을 기반으로 구축되었다는 점이 가장 중요한 공통점이다.¹ 이로 인해 두 보드는 동일한 듀얼 코어 ARM Cortex-A9 프로세싱 시스템(PS)과 Artix-7 프로그래머블 로직(PL)을 공유하며, 85k 로직 셀과 220개의 DSP 슬라이스와 같은 동일한 연산 능력을 갖추고 있다. 이 공통 기반이 ANTSDR E310으로의 PYNQ 포팅을 기술적으로 가능하게 하는 핵심 요소이다.
- **DDR3 메모리:** 메모리 용량에서 첫 번째 중요한 차이가 나타난다. ANTSDR E310은 일반적으로 두 개의 512MB 칩으로 구성된 총 **1 GByte**의 **DDR3 RAM**을 탑재하고 있다.² 반면, PYNQ-Z2는 **512 MByte**의 **DDR3 RAM**을 제공한다.³ SDR 애플리케이션은 대규모 I/Q 샘플 데이터를 버퍼링하여 처리하는 경우가 많기 때문에, ANTSDR E310의 더 큰 메모리 용량은 고대역폭 신호 처리 시 상당한 이점을 제공한다.
- **플래시 메모리:** ANTSDR E310은 **32 MB**의 **QSPI** 플래시를 포함하는 반면,² PYNQ-Z2는

16 MB (128 Mbit)의 **Quad-SPI** 플래시를 갖추고 있다.³ 이는 부트로더, 환경 변수 및 기타 비휘발성 데이터를 저장하는 용량에 영향을 미친다.

- **부팅 매체:** 두 보드 모두 PYNQ의 표준 배포 방식인 **MicroSD** 카드를 주 부팅 매체로 사용한다.¹⁰ 추가적으로 ANTSDR은 점퍼 설정을 통해 **QSPI** 플래시 부팅 및 **DFU(Device Firmware Update)**를 명시적으로 지원하여 임베디드 환경에서의 유연성을 높였다.¹²

1.2. 무선 주파수(RF) 및 아날로그 프런트엔드

두 보드의 가장 근본적인 차이점은 RF 기능의 유무이다.

- **ANTSDR E310:** 이 보드의 핵심 정체성은 내장된 RF 서브시스템에 있다. **Analog Devices**사의 **AD9361** 고성능 RF 트랜시버를 사용하여 **2x2 MIMO(2 RX, 2 TX)** 구성을 지원한다.¹ 이 단일 칩은 **70 MHz**에서 **6 GHz**에 이르는 넓은 주파수 범위를 커버하며, 최대 **56 MHz**의 순간 대역폭을 제공한다.¹ 이 보드는 **ADALM-PLUTO**에서 영감을 받아 설계되었지만, 더 강력한 **AD9361** 칩과 **Zynq-7020 SoC**를 채택하여 성능을 향상시켰다.² 중요한 점은, 이름이 비슷한 **Ettus Research**의 **USRP E310**이 동적으로 전환되는 정교한 하드웨어 필터 बैं크를 갖춘 것과 달리,¹ **ANTSDR**의 RF 프런트엔드는 **ADI**의 **FMCOMMS3** 레퍼런스 디자인과 유사한 광대역 플랫폼에 가깝다는 것이다.⁵
- **PYNQ-Z2:** PYNQ-Z2는 내장된 **RF** 프런트엔드가 전혀 없다. 이 보드는 순수한 디지털 로직 및 프로세싱 플랫폼이다. RF 기능은 **Pmod**나 **Arduino** 헤더와 같은 확장 커넥터를 통해 외부 모듈을 연결해야만 구현할 수 있다. 이는 PYNQ-Z2가 고성능 **SDR** 애플리케이션을 즉시 수행하기에는 부적합하며, 그 설계 철학이 **ANTSDR**과 근본적으로 다름을 보여준다.

1.3. 주변 장치 연결성 및 I/O 인터페이스

두 보드는 연결성 측면에서 각자의 설계 목표에 따라 뚜렷한 차이를 보인다.

- **이더넷:** 두 보드 모두 **기가비트 이더넷** 포트를 갖추고 있다.² 그러나 구현 방식에 결정적인 차이가 있다. PYNQ-Z2는 **Zynq PS**에 내장된 **기가비트 이더넷 MAC(GEM)** 중 하나를 외부 **PHY** 칩에 연결하여 사용한다.³ 반면, **ANTSDR E310**은 개발자 문서 및 관련 커뮤니티 저장소 분석 결과¹⁷, ****프로그래머블 로직(PL)****에 **AXI Ethernet IP** 코어를 구현하여 이더넷 인터페이스를 구성한다. 이는 이더넷 **MAC**이 네이티브 **PS** 주변 장치가 아니므로, **Vivado** 블록 디자인과 디바이스 트리 구성에 지대한 영향을 미친다.
- **디버그 인터페이스:** **ANTSDR E310**은 통합된 **USB-JTAG** 및 **USB-UART** 인터페이스를 제공하여 개발 편의성을 크게 높였다.² PYNQ-Z2는 단일 **Micro USB** 포트를 통해 **JTAG**

- 프로그래밍과 UART 통신을 모두 처리하며, 이는 브리지 칩에 의해 관리된다.³
- **범용 I/O:** PYNQ-Z2는 두 개의 Pmod 포트, Arduino 쉴드 커넥터, Raspberry Pi 커넥터 등 풍부한 확장 옵션을 제공하여 이 분야에서 탁월하다.³ ANTSDR은 RF 커넥터와 핵심 인터페이스에 집중하여 상대적으로 단출하지만, GPS 동기화를 위한 확장 포트가 언급되어 있다.²
 - **멀티미디어:** PYNQ-Z2는 HDMI 입출력 포트와 오디오 코덱을 포함하여 비디오 처리 및 일반 임베디드 시스템 프로젝트에 적합하다.³ ANTSDR E310에는 이러한 멀티미디어 중심의 주변 장치가 전혀 없다.

1.4. 클럭, 전원 및 물리적 특성

- **클럭:** ANTSDR E310은 무선 통신에서 주파수 정확도를 위해 필수적인 고정밀 **0.5ppm TCXO(온도 보상 수정 발진기)**를 특징으로 한다.² 이는 주파수 드리프트를 최소화하는 전문가급 사양이다. PYNQ-Z2는 디지털 로직에 충분한 표준 발진기(예: PS용 50 MHz, PL용 125 MHz)를 사용하며, 정밀한 RF 작업을 위해서는 외부 기준 클럭이 필요하다.²⁰
- **전원:** ANTSDR E310은 일반적으로 USB OTG 포트를 통해 **USB** 전원으로 동작한다.² PYNQ-Z2는 USB 또는 외부 7V-15V 전원 어댑터를 사용할 수 있으며, 점퍼로 전원 소스를 선택한다.³

이러한 하드웨어 분석은 두 보드의 설계 철학이 근본적으로 다르다는 것을 명확히 보여준다. PYNQ-Z2는 다양한 I/O 유연성을 강조하는 다목적 교육용 플랫폼인 반면,³ ANTSDR E310은 ADALM-PLUTO의 비용 효율적인 고성능 대안으로 설계된 특수 목적의 SDR 개발 플랫폼이다.² 메모리 크기, RF 통합, TCXO 정밀도, PL 기반 이더넷과 같은 모든 핵심 차이점은 이러한 근본적인 설계 의도에서 비롯된다.

특히 ANTSDR의 이더넷이 PL에 구현되어 있다는 점은 포팅 과정에서 가장 중요한 기술적 고려사항이다. 표준 PYNQ 이미지는 PS 기반 이더넷(PS-GEM)을 가정하고 설계되었다. 따라서 ANTSDR로의 포팅은 단순히 AD9361 드라이버를 추가하는 것을 넘어, `axi_ethernet` 코어를 포함하고 구성하도록 기본 Vivado 디자인과 디바이스 트리를 근본적으로 수정해야 함을 의미한다. 이는 포팅의 복잡성을 크게 증가시키는 요인이다.

또한, ANTSDR이 ADALM-PLUTO에서 영감을 받았다는 점은 중요한 기술적 단서를 제공한다.² 이는 Analog Devices가 Pluto를 위해 구축한 전체 소프트웨어 스택, 즉 IIO 드라이버를 포함한 리눅스 커널,

`libiio`, `pyadi-iio`가 ANTSDR에 적용할 올바른 아키텍처임을 강력하게 시사한다.²⁵ 따라서 포팅 작업의 본질은 새로운 드라이버 스택을 개발하는 것이 아니라, 기존의 검증된 ADI 스택을 PYNQ 빌드 시스템에 통합하는 것이다.

기능	ANTSDR E310	PYNQ-Z2
SoC	Xilinx Zynq XC7Z020-1CLG400C ²	Xilinx Zynq XC7Z020-1CLG400C ³
프로세서	듀얼 코어 ARM Cortex-A9 @ 766 MHz ²	듀얼 코어 ARM Cortex-A9 @ 650 MHz ³
DDR3 RAM	1 GByte (2x 512MB) ²	512 MByte ³
QSPI 플래시	32 MByte ²	16 MByte (128 Mbit) ³
RF 트랜시버	Analog Devices AD9361 ¹	없음
주파수 범위	70 MHz – 6 GHz ¹	해당 없음
순간 대역폭	최대 56 MHz ¹	해당 없음
MIMO 구성	2x2 (2 RX, 2 TX) ¹	해당 없음
이더넷	10/100/1000 Mbps (PL 기반 AXI Ethernet) ²	10/100/1000 Mbps (PS 기반 GEM) ³
디버그	통합 USB-JTAG 및 USB-UART ²	Micro USB (JTAG + UART 브리지) ³
확장 I/O	확장 포트 (GPS 동기화용) ²	2x Pmod, 1x Arduino, 1x Raspberry Pi ³
멀티미디어	없음	HDMI 입/출력, 오디오 코덱 ³
기준 클럭	0.5ppm TCXO ²	표준 발진기 (예: 50 MHz) ²⁰

섹션 2: 커스텀 **SDR** 하드웨어로의 **PYNQ** 포팅을 위한 핵심 개념

본 섹션에서는 하드웨어 분석과 실제 포팅 가이드를 연결하는 데 필요한 이론적 배경을 제공한다. 이는 섹션 3에서 사용될 도구와 프레임워크의 '무엇'과 '왜'를 설명하여, 각 단계의 기술적 당위성을 명확히 하는 것을 목표로 한다.

2.1. PYNQ v3.1 빌드 환경 (sdbuild)

PYNQ SD 카드 이미지는 전통적인 임베디드 리눅스 배포판과 다른 고유한 빌드 시스템을 사용한다.

- **개요:** PYNQ SD 카드 이미지는 PYNQ GitHub 저장소의 **sdbuild** 디렉터리에 위치한 커스텀 빌드 시스템을 통해 생성된다.²⁷ 이 시스템은 **PetaLinux**, **Vivado**, 그리고 **Ubuntu** 루트 파일 시스템 빌드 프로세스를 통합적으로 관리하고 조율하는 역할을 한다.
- **Docker 기반 워크플로우:** PYNQ v3.1부터는 **Docker** 컨테이너를 사용하는 것이 권장 빌드 환경이다.²⁸ **Docker**는 의존성 및 환경 구성을 격리된 컨테이너 내에서 관리함으로써 호스트 OS의 차이로 인한 문제를 해결하고, 일관성 있고 재현 가능한 빌드 환경을 보장한다. 본 가이드에서는 이 방법을 기본으로 채택한다.
- **보드 명세 파일 (<BOARD>.spec):** 이 파일은 새로운 보드를 타겟팅하기 위한 핵심 설정 파일이다. 이 파일 내에는 대상 아키텍처(**ARCH**), 사용할 **PetaLinux BSP(BSP_<BOARD>)**, 그리고 설치할 커스텀 소프트웨어 패키지(**STAGE4_PACKAGES_<BOARD>**)와 같은 중요한 변수들이 정의된다.²⁸ 포팅 과정의 핵심 과제 중 하나는 **ANTSDR E310**에 맞는 **antsdr-e310.spec** 파일을 작성하는 것이다.
- **PetaLinux 통합:** PYNQ의 **sdbuild**는 내부적으로 **PetaLinux**를 호출하여 부트로더(**FSBL**, **U-Boot**), 리눅스 커널, 그리고 디바이스 트리를 빌드한다.³¹ 따라서 포팅 작업의 핵심은 **sdbuild**가 생성하는 **PetaLinux** 프로젝트를 **ANTSDR** 하드웨어에 맞게 올바르게 설정하는 것이다.

2.2. Analog Devices IIO 서브시스템

Analog Devices의 최신 트랜시버 칩을 리눅스 환경에서 사용하기 위해서는 **IIO(Industrial I/O)** 서브시스템에 대한 이해가 필수적이다.

- **목적:** 리눅스 **IIO** 서브시스템은 센서, **ADC**, **DAC**와 같은 아날로그 인터페이스 장치를 위한 표준 커널 프레임워크이다. **Analog Devices**는 **AD9361**을 포함한 자사의 모든 최신 트랜시버 칩을 지원하기 위해 이 프레임워크를 적극적으로 활용하고 기여하고 있다.³³
- **meta-adi 레이어:** 통합 과정을 단순화하기 위해, **ADI**는 **Yocto/PetaLinux**용 레이어인 **meta-adi-core**와 **meta-adi-xilinx**를 제공한다.³³ 이 레이어들에는 **ADI** 하드웨어를 지원하는 데 필요한 커널 모듈, 디바이스 트리 오버레이, 그리고 사용자 공간 라이브러리(

libiio)를 빌드하기 위한 레시피가 포함되어 있다. PetaLinux 빌드에 이 레이어들을 추가하는 것은 필수적인 단계이다.

- **libiio와 pyadi-iio:** libiio는 애플리케이션에서 IIO 장치와 상호작용하기 위한 API를 제공하는 사용자 공간 C 라이브러리이다.³⁴
pyadi-iio는 libiio를 감싼 파이썬 래퍼(wrapper)로, 특히 ADI 부품에 대해 높은 수준의 객체 지향 인터페이스를 제공한다.²⁵ PYNQ 포팅의 궁극적인 목표는 이 pyadi-iio가 Jupyter 환경 내에서 원활하게 동작하도록 만드는 것이다.

2.3. 하드웨어-소프트웨어 계약으로서의 디바이스 트리

디바이스 트리는 부팅 시 리눅스 커널에 전달되는 데이터 구조로, 시스템의 하드웨어 구성을 설명한다.¹⁰ 이를 통해 단일 커널 바이너리가 다양한 하드웨어 구성에서 실행될 수 있다. 포팅 과정에서 디바이스 트리는 리눅스에게 ANTSDR의 하드웨어 구성을 명시적으로 알려주는 '계약서' 역할을 한다.

- **ANTSDR을 위한 핵심 노드:** 포팅 과정에서는 다음과 같은 여러 핵심 노드를 정의하거나 수정해야 한다.
 - **SPI 컨트롤러 노드:** AD9361이 물리적으로 연결된 PL의 AXI QSPI 컨트롤러를 정의한다.
 - **클럭 노드:** 고정밀 TCXO를 고정 클럭 소스로 정의한다.
 - **AD9361 PHY 노드:** 가장 중요한 노드이다. 이 노드는 **compatible** 속성으로 "adi,ad9361-phy"를 가지며, 트랜시버의 동작 모드(예: MIMO 구성, 디지털 인터페이스 튜닝)를 설정하기 위한 수많은 **adi,*** 속성을 포함하게 된다.¹⁸
 - **AXI 이더넷 노드:** PL 기반 이더넷 MAC과 외부 PHY 칩과의 연결을 정의한다.

이러한 개념들을 종합해 볼 때, 포팅 과정은 단일 작업이 아닌 다층적 통합 문제임을 알 수 있다. 최상위에는 PYNQ의 **sdbuild** 시스템이 있고, 이는 PetaLinux 레이어를 제어한다. PetaLinux 레이어는 **meta-adi** 레이어를 포함하도록 설정되어야 하며, **meta-adi** 레이어는 Vivado 하드웨어 디자인에 기반하여 생성된 디바이스 트리 레이어에 의해 구성되는 커널 드라이버를 제공한다. 어느 한 계층에서의 실패는 전체 프로세스의 실패로 이어지므로, 가이드는 각 계층을 올바른 순서로 체계적으로 다루어야 한다.

또한, **sdbuild** 시스템은 루트 파일 시스템 생성과 최종 이미지 패키징을 자동화하여 도움을 주지만²⁸, 동시에 PetaLinux 프로젝트 위에 추상화 계층을 추가하여 문제를 복잡하게 만들기도 한다. 빌드 실패 시 디버깅을 위해서는

sdbuild가 내부적으로 PetaLinux를 어떻게 호출하는지 이해해야 한다. 따라서 성공적인 포팅 가이드는 단순히 성공 경로의 명령어만 제공하는 것이 아니라, **sdbuild**가 생성하는 중간 빌드 디렉토리를 검사하여 빌드 실패를 디버깅하는 방법에 대한 실질적인 조언도 포함해야 한다.

섹션 3: ANTSDR E310으로 PYNQ v3.1을 포팅하기 위한 단계별 가이드

본 섹션은 보고서의 핵심으로, 실제 실행 가능한 명령어 수준의 지침을 제공한다. 각 단계는 그 목적과 하드웨어 분석과의 연관성에 대한 설명을 동반한다.

3.1. 환경 설정 및 소스 준비

성공적인 빌드를 위해서는 정확한 버전의 도구와 소스 코드를 준비하는 것이 첫걸음이다.

- **호스트 시스템:** Ubuntu 22.04 LTS를 실행하는 리눅스 호스트 시스템이 필요하다.
- **AMD 툴체인 설치:** PYNQ v3.1 빌드에는 **Vivado/Vitis** 및 **PetaLinux v2024.1** 버전이 필요하다.²⁸ 이는 기존 ANTSDR 관련 가이드들이 명시한 구버전(예: 2019.1)과 다른 중요한 차이점이므로 반드시 최신 버전을 설치해야 한다.¹³
- **Docker 설정:** PYNQ v3.1 공식 문서에 따라 Docker를 설치하고, PYNQ 빌드 환경을 위한 Docker 이미지를 생성한다.²⁸ 이는 호스트 시스템의 의존성 문제를 최소화하고 재현 가능한 빌드를 보장한다.
- **소스 코드 복제:** PYNQ v3.1 저장소와 ADI HDL 저장소를 복제한다. ADI HDL 저장소에는 axi_ad9361과 같은 필수 IP 코어가 포함되어 있다.

Bash

```
git clone --recursive https://github.com/Xilinx/PYNQ.git
```

```
git clone https://github.com/analogdevicesinc/hdl.git
```

3.2. ANTSDR PYNQ 보드 정의 생성

sdbuild 시스템이 ANTSDR E310을 인식하고 타겟팅할 수 있도록 새로운 보드 정의를 생성한다.

- **디렉터리 구조 생성:** PYNQ 저장소 내에 새로운 보드 디렉터를 생성한다.

Bash

```
mkdir -p <PYNQ_ROOT>/sdbuild/boards/antsdr-e310
```

- **antsdr-e310.spec 파일 작성:** 생성한 디렉터리 내에 보드 명세 파일을 작성한다. 이 파일은 빌드 시스템에 타겟 아키텍처와 사용할 BSP의 이름을 알려준다. 초기에는 BSP 파일이 없지만, 다음 단계에서 생성될 것이다.

Makefile

```
# <PYNQ_ROOT>/sdbuild/boards/antsdr-e310/antsdr-e310.spec
```

```
ARCH_antsdr-e310 := arm
```

```
BSP_antsdr-e310 := antsdr-e310.bsp
```

```
BITSTREAM_antsdr-e310 :=
```

- **Vivado** 프로젝트 생성 (**TCL** 스크립트): `antsdr-e310.tcl` 스크립트를 작성하여 하드웨어 플랫폼을 정의한다. 이 스크립트는 다음 작업을 자동화한다.
 1. `xc7z020clg400-1` 디바이스를 타겟으로 **Vivado** 프로젝트를 생성한다.
 2. Zynq7 Processing System IP를 인스턴스화하고, 기존 **ANTSDR** 프로젝트(예: `antsdr-fw` 저장소)를 참조하여 DDR 타이밍 및 활성화된 PS 주변 장치(UART, SDIO 등)를 설정한다.¹³
 3. 복제한 ADI HDL 라이브러리 경로를 프로젝트에 추가한다.
 4. 핵심 IP인 `axi_ad9361` (ADI)과 `axi_ethernet` (Xilinx)을 인스턴스화한다.
 5. Vivado의 Block Automation 기능을 사용하여 이 IP들의 AXI 인터페이스를 Zynq PS에 연결한다.
 6. RF 데이터(LVDS), SPI, 이더넷 RGMII 등 필요한 포트들을 외부 포트로 만든다.
 7. 블록 디자인을 검증하고 HDL 래퍼를 생성한다.
- **PetaLinux** BSP 생성: `petalinux_bsp` 하위 디렉토리를 만들고, 그 안에 3.4절에서 설명할 `system-user.dtsi` 파일과 커널 설정 조각 파일을 위치시킨다. `sdbuild`의 `make` 프로세스는 이 파일들을 사용하여 보드에 특화된 **BSP**를 자동으로 생성한다.

3.3. ADI IIO 스택을 위한 PetaLinux 커스터마이징

`sdbuild`가 내부적으로 사용하는 **PetaLinux** 프로젝트에 ADI 관련 드라이버와 라이브러리를 추가하도록 설정한다.

- **meta-adi** 통합: **PetaLinux** 빌드에 `meta-adi` 레이어를 추가해야 한다. 이는 보드 디렉토리 내에 **PetaLinux** 설정 조각 파일(`petalinux-config-fragment`)을 생성하여 사용자 레이어 설정에 `meta-adi-core`와 `meta-adi-xilinx`의 경로를 추가하는 방식으로 이루어진다.
- 커널 설정: 필요한 커널 드라이버를 활성화하기 위해 `kernel-config-fragment` 파일을 제공한다. 이 파일은 보드의 **PetaLinux** 레시피 디렉토리에 위치하며, 다음과 같은 설정들을 포함해야 한다. 이는 ³³에서 제공된 템플릿을 기반으로 한다.

```
CONFIG_IIO=y
```

```
CONFIG_IIO_BUFFER=y
```

```
CONFIG_IIO_TRIGGER=y
```

```
CONFIG_AD9361=y
```

```
CONFIG_AXI_SPI_ENGINE=y
```

```
CONFIG_AXI_ETHERNET=y
```

```
#... 기타 IIO 및 ADI 드라이버 관련 의존성 설정
```


3.4. ANTSDR 하드웨어를 위한 디바이스 트리 수정

디바이스 트리는 리눅스 커널이 ANTSDR의 하드웨어 구성, 특히 AD9361과 PL 기반 이더넷을 인식하게 하는 가장 중요한 부분이다.

- **system-user.dtsi** 작성: 이 파일은 antsd-r-pynq 저장소의 pl.dts¹⁸와 ADI의 공식 문서³³를 참조하여 작성한다. 이 파일은 PetaLinux가 생성하는 기본 디바이스 트리에 추가적인 하드웨어 정보를 오버레이한다.
- 클럭 정의: 40MHz TCXO를 fixed-clock으로 정의하여 AD9361의 기준 클럭으로 사용될 수 있도록 한다.
- **AXI SPI** 노드: Vivado 디자인에서 AD9361 제어에 사용되는 AXI SPI 컨트롤러 인스턴스를 정의한다.
- **AD9361 PHY** 노드: 디바이스 트리의 핵심 부분이다. 아래 예시와 같이 작성되며, 각 속성에 대한 설명은 이어지는 표에 정리되어 있다.

DTS

```
&spi0 { // Vivado의 AXI SPI 컨트롤러 인스턴스와 일치해야 함
    status = "okay";
    ad9361-phy@0 {
        compatible = "adi,ad9361-phy";
        reg = ;
        #clock-cells = ;
        clocks = <&ad9361_clkin>; // TCXO 고정 클럭 노드를 참조
        clock-names = "ad9361_ext_refclk";
        spi-cpha;
        spi-max-frequency = ;

        /* ANTSDR을 위한 핵심 속성들 */
        adi,rx-2tx-mode-enable; // 2x2 MIMO 모드 활성화
        adi,lvds-mode-enable; // LVDS 디지털 인터페이스 사용
        adi,rf-rx-bandwidth-hz = ; // 수신 RF 대역폭 (예: 18MHz)
        adi,rf-tx-bandwidth-hz = ; // 송신 RF 대역폭 (예: 18MHz)
        adi,tx-attenuation-mdB = ; // 초기 송신 감쇠 (10dB)
        /* 기타 [33] 및 [18]에서 참조한 속성들... */
    };
};
```

- **AXI** 이더넷 노드: axi_ethernet 노드를 정의하고, phy-handle를 통해 외부 PHY 정의를 가리키며 phy-mode를 "rgmii-id"로 설정한다.

속성 이름	설명 및 목적
compatible	커널이 이 노드에 바인딩할 올바른 드라이버("adi,ad9361-phy")를 식별하도록 한다.
clocks	AD9361 칩에 공급되는 외부 기준 클럭의 핸들을 지정한다.
adi,2rx-2tx-mode-enable	칩을 2개의 수신 채널과 2개의 송신 채널을 사용하는 MIMO 모드로 설정한다.
adi,lvds-mode-enable	PS-PL 간 데이터 전송을 위해 LVDS(저전압 차분 신호) 디지털 인터페이스를 사용하도록 설정한다.
adi,rf-rx-bandwidth-hz	수신 경로의 아날로그 필터 대역폭을 헤르츠 단위로 설정한다.
adi,digital-interface-tune-skip-mode	디지털 인터페이스 튜닝 절차의 동작을 제어한다. (0: RX/TX 모두 튜닝, 1: TX 스킵, 2: 모두 스킵)

3.5. SD 카드 이미지 빌드 및 배포

모든 설정이 완료되면, `sdbuild`를 사용하여 최종 SD 카드 이미지를 생성한다.

- 빌드 명령어: `<PYNQ_ROOT>/sdbuild` 디렉터리에서 다음 명령어를 실행한다. 이 과정은 시스템 사양에 따라 수 시간이 소요될 수 있다.

```
Bash
make BOARDS=antsdr-e310
```
- 이미지 위치: 빌드가 성공적으로 완료되면, 최종 이미지는 `<PYNQ_ROOT>/sdbuild/output/antsdr-e310-v3.1.img` 경로에 생성된다.
- SD 카드에 쓰기: BalenaEtcher나 `dd`와 같은 도구를 사용하여 생성된 이미지를 MicroSD 카드에 기록한다.
- 보드 설정: ANTSDR 보드의 부팅 모드 점퍼를 "SD card"로 설정하고 ¹⁰, 전원, UART, 이더넷

케이블을 연결한다.

3.6. 시스템 검증 및 디버깅

이미지가 부팅된 후, 포팅이 성공적으로 이루어졌는지 확인하는 단계이다.

- **시리얼 콘솔:** USB-UART에 연결하여 부팅 메시지를 모니터링한다. 부팅 로그에서 AD9361 및 AXI 이더넷 드라이버가 성공적으로 프로브(probe)되는지 확인한다.
- **커널 로그 (dmesg):** 부팅 후 보드에서 `dmesg | grep -i "ad9361\|axi_eth"` 명령어를 실행하여 관련 드라이버가 오류 없이 로드되었는지 확인한다.
- **IIO 장치 확인:** `ls /sys/bus/iio/devices/`를 실행하여 ad9361-phy에 해당하는 iio:deviceX가 생성되었는지 확인한다. 또한, libiio가 설치된 호스트 PC에서 `iio_info -n <보드_IP_주소>`를 실행하여 IIO 데몬과의 네트워크 통신을 검증한다.
- **네트워크 확인:** 보드에서 `ifconfig`를 실행하여 이더넷 인터페이스가 IP 주소를 할당받았는지 확인하고, `ping` 명령어로 네트워크 게이트웨이와 통신하여 연결성을 최종 확인한다.

섹션 4: 애플리케이션 및 검증: 포팅된 PYNQ 시스템 사용

이 마지막 섹션에서는 파이썬/Jupyter 계층에서 RF 하드웨어까지의 엔드투엔드 기능 시연을 통해 포팅의 성공을 증명한다. 복잡한 저수준 포팅 작업이 완료된 후, PYNQ 프레임워크가 제공하는 고수준 추상화의 강력함이 드러난다. 디바이스 트리의 복잡한 설정과 대조적으로, 파이썬 코드 몇 줄로 하드웨어를 제어할 수 있게 되는 것은 PYNQ가 제공하는 생산성 향상의 핵심 가치를 보여준다.

4.1. pyadi-iio를 이용한 AD9361 초기화

Jupyter 환경에서 pyadi-iio 라이브러리를 사용하여 ANTSDR의 AD9361 트랜시버와 통신하는 첫 단계이다.

- **Jupyter에 연결:** 호스트 PC의 웹 브라우저에서 `http://pynq:9090` 또는 보드의 IP 주소로 접속하여 Jupyter Notebook 환경에 로그인한다.
- **Jupyter Notebook 예제:** 새 노트북을 생성하고 다음 코드를 실행하여 하드웨어와의

통신을 확인한다. 이 코드는 pyadi-iio 예제를 기반으로 한다.²⁵

```
Python
import adi

# 네트워크 설정에 따라 IP 주소가 달라질 수 있음
sdr_uri = "ip:192.168.2.1"

try:
    sdr = adi.ad9361(uri=sdr_uri)
    print("ANTSDR E310에 성공적으로 연결되었습니다!")
    print(f"수신 LO 주파수: {sdr.rx_lo} Hz")
    print(f"송신 LO 주파수: {sdr.tx_lo} Hz")
except Exception as e:
    print(f"SDR 연결 오류: {e}")
```

이 코드가 오류 없이 실행되고 현재 설정된 LO 주파수가 출력된다면, PYNQ 소프트웨어 스택이 IIO 드라이버를 통해 AD9361 칩과 성공적으로 통신하고 있음을 의미한다.

4.2. 개념 증명: 간단한 스펙트럼 분석기

실제 SDR 작업을 수행하는 완전한 Jupyter Notebook 예제를 통해 전체 신호 체인의 기능성을 시각적으로 검증한다.

- 전체 노트북 예제: 다음은 수신된 신호의 스펙트럼을 분석하고 플로팅하는 전체 코드이다.

```
Python
import adi
import numpy as np
import matplotlib.pyplot as plt

# 1. SDR 장치에 연결
sdr_uri = "ip:192.168.2.1"
sdr = adi.ad9361(uri=sdr_uri)

# 2. SDR 파라미터 설정
sdr.sample_rate = int(2.4e6) # 샘플링 속도: 2.4 MSPS
sdr.rx_rf_bandwidth = int(2e6) # RF 대역폭: 2 MHz
sdr.rx_lo = 98500000 # 수신 중심 주파수 (예: 98.5 MHz FM 방송)
sdr.gain_control_mode_chan0 = 'slow_attack' # 자동 이득 제어 활성화
sdr.rx_buffer_size = 2**16 # 수신 버퍼 크기

# 3. 데이터 수신
print("데이터 수신 중...")
```

```

rx_samples = sdr.rx()
print("데이터 수신 완료.")

# 4. 신호 처리 및 플로팅
plt.style.use('dark_background')
plt.figure(figsize=(12, 6))

# Power Spectral Density (PSD) 계산
plt.psd(rx_samples, NFFT=1024, Fs=sdr.sample_rate / 1e6, Fc=sdr.rx_lo / 1e6)

plt.title("수신 신호의 주파수 스펙트럼")
plt.xlabel("주파수 (MHz)")
plt.ylabel("상대 전력 (dB/Hz)")
plt.grid(True)
plt.show()

```

- **실행 및 결과:** 이 노트북을 실행하면 ANTSDR E310은 98.5 MHz를 중심으로 신호를 수신하고, 수집된 I/Q 샘플은 numpy를 통해 FFT 처리된 후 matplotlib을 통해 파워 스펙트럼으로 시각화된다. 안테나가 연결된 상태에서 이 코드를 실행하면, 해당 주파수 대역의 지역 FM 라디오 방송 신호가 스펙트럼 플롯에 피크 형태로 나타나는 것을 확인할 수 있다. 이러한 시각적 결과는 파이썬 API부터 RF 프런트엔드, ADC, PS-PL 인터페이스, DMA, 그리고 커널 드라이버에 이르는 전체 신호 경로가 올바르게 작동하고 있음을 최종적으로 증명한다.

결론

본 보고서는 ANTSDR E310과 PYNQ-Z2의 하드웨어 아키텍처를 엄밀하게 비교 분석하고, 이를 바탕으로 PYNQ v3.1 프레임워크를 ANTSDR E310에 포팅하기 위한 상세한 단계별 가이드를 제시했다.

분석 결과, 두 보드는 동일한 Zynq-7020 SoC를 공유하여 포팅의 기반을 제공하지만, 설계 철학의 차이로 인해 핵심 구성 요소에서 중요한 차이점을 보였다. ANTSDR E310은 1 GByte의 더 큰 DDR3 메모리, 고성능 AD9361 RF 트랜시버, 그리고 고정밀 TCXO를 탑재하여 SDR 애플리케이션에 최적화된 반면, PYNQ-Z2는 다양한 확장 I/O와 멀티미디어 인터페이스를 갖춘 범용 교육 및 프로토타이핑 플랫폼이다. 특히, ANTSDR의 이더넷 인터페이스가 Zynq의 프로그래머블 로직(PL)에 구현되어 있다는 점은 표준 PYNQ 이미지의 가정과 달라 포팅 과정의 복잡성을 가중시키는 핵심 요인으로 식별되었다.

제시된 포팅 가이드는 이러한 하드웨어적 차이를 극복하기 위해 체계적인 접근법을 채택했다.

PYNQ v3.1의 Docker 기반 빌드 환경을 시작으로, ANTSDR 하드웨어(특히 PL 기반 이더넷과 AD9361)를 정의하는 Vivado 프로젝트 생성, Analog Devices의 meta-adi 레이어를 PetaLinux

빌드에 통합하여 IIO 드라이버 스택을 구축하는 과정, 그리고 하드웨어와 소프트웨어를 연결하는 디바이스 트리 파일을 정밀하게 작성하는 방법을 상세히 기술했다.

최종적으로, 포팅된 시스템에서 pyadi-iio 파이썬 라이브러리를 사용하여 Jupyter Notebook 환경에서 ANTSDR의 RF 기능을 성공적으로 제어하고, 실제 FM 방송 신호를 수신하여 스펙트럼을 분석하는 개념 증명 애플리케이션을 통해 전체 포팅 과정의 유효성을 입증했다.

결론적으로, ANTSDR E310으로의 PYNQ 3.1 포팅은 상당한 기술적 노력을 요구하는 복잡한 과정이지만, 본 가이드에서 제시한 절차를 통해 성공적으로 달성할 수 있다. 이 통합을 통해 개발자는 ANTSDR E310의 강력한 RF 하드웨어 성능과 PYNQ 프레임워크가 제공하는 높은 수준의 생산성 및 유연성을 결합하여, 차세대 무선 통신 시스템의 연구 및 프로토타이핑을 가속화할 수 있는 강력한 개발 플랫폼을 확보하게 된다.

참고 자료

1. USRP E310 Datasheet - Ettus Research, 9월 30, 2025에 액세스, https://www.ettus.com/wp-content/uploads/2019/01/USRP_E310_Datasheet.pdf
2. MicroPhase ANTSDR E310 AD9361 Software Defined Radio Development Board ZYNQ7020- | eBay, 9월 30, 2025에 액세스, <https://www.ebay.com/itm/126254547358>
3. PYNQ-Z2 Development Board - uri=media.digikey, 9월 30, 2025에 액세스, https://media.digikey.com/pdf/Data%20Sheets/DFRobot%20PDFs/DFR0600_Web.pdf
4. Pynq-Z2 - TUL Embedded, 9월 30, 2025에 액세스, <https://www.tulembedded.com/fpga/ProductsPYNQ-Z2.html>
5. MicroPhase ANTSDR E310 AD9361 Software Defined Radio Open Source SDR ZYNQ7020, 9월 30, 2025에 액세스, <https://www.ebay.com/itm/296798053980>
6. MicroPhase ANTSDR E310 Software Defined Radio Demo Board transceiver ZYNQ 7000 SoC ADI AD9361 SDR transmitter and receiver - AliExpress, 9월 30, 2025에 액세스, <https://www.aliexpress.com/item/1005006638962896.html>
7. AUP PYNQ-Z2 - AMD, 9월 30, 2025에 액세스, <https://www.amd.com/en/corporate/university-program/aup-boards/pynq-z2.html>
8. PYNQ-Z2 | Real Digital, 9월 30, 2025에 액세스, <https://www.realdigital.org/hardware/pynq-z2>
9. Introducing TUL PYNQTM-Z2 Basic kit - SPARKLE Embedded, 9월 30, 2025에 액세스, <https://www.sparkle-embedded.com.tw/files/20240731143029314.pdf>
10. sunleechina/antsdr-fw - Gitee, 9월 30, 2025에 액세스, <https://gitee.com/sunleechina/antsdr-fw>
11. PYNQ-Z2 Setup Guide — Python productivity for Zynq (Pynq) - Read the Docs, 9월 30, 2025에 액세스, https://pynq.readthedocs.io/en/v2.7.0/getting_started/pynq_z2_setup.html
12. bkerler/antsdr_new: Latest firmware for antsdr E310 based on PlutoSDR - GitHub, 9월 30, 2025에 액세스, https://github.com/bkerler/antsdr_new/
13. MicroPhase/antsdr-fw-patch: Repository of antsdr firmware make - GitHub, 9월

- 30, 2025에 액세스, <https://github.com/MicroPhase/antsdr-fw-patch>
14. E310/E312 - Ettus Knowledge Base, 9월 30, 2025에 액세스, <https://kb.ettus.com/E310/E312>
 15. ANTSDR E310 Features - Satsagen - Groups.io, 9월 30, 2025에 액세스, https://groups.io/g/satsagen/topic/antsdr_e310_features/96870466
 16. AD-FMCOMMS3-EBZ Board 9 - AD9361 RF Agile Transceiver - Fraser Innovation Inc, 9월 30, 2025에 액세스, <https://fraserinnovations.com/product/fii-ad9361/>
 17. MicroPhase/antsdr-pynq - GitHub, 9월 30, 2025에 액세스, <https://github.com/MicroPhase/antsdr-pynq>
 18. 1월 1, 1970에 액세스, https://raw.githubusercontent.com/MicroPhase/antsdr-pynq/main/PYNQ-PRIO/boards/e200/prio_linux/dtso/pl.dts
 19. PYNQ-Z2 Board Features, Applications, and Advantages, 9월 30, 2025에 액세스, <https://www.ariat-tech.com/blog/PYNQ-Z2-Board-Features,Applications,and-Advantages.html>
 20. PYNQ Board Overview, 9월 30, 2025에 액세스, <https://pynq.tue.nl/general/pynq/>
 21. AUP PYNQ-Z2 - AMD, 9월 30, 2025에 액세스, <https://www.amd.com/fr/corporate/university-program/aup-boards/pynq-z2.html>
 22. MicroPhase ANTSDR E310 AD9363 SDR Development Board for ADI Pluto Communications | eBay, 9월 30, 2025에 액세스, <https://www.ebay.com/itm/355222519305>
 23. PYNQ | Python Productivity for AMD Adaptive Computing platforms, 9월 30, 2025에 액세스, <http://www.pynq.io/>
 24. MicroPhase ANTSDR E310 AD9363 SDR Development Board for ADI Pluto Communications | eBay, 9월 30, 2025에 액세스, <https://www.ebay.com/itm/364270363371>
 25. Examples — Analog Devices Hardware Python Interfaces 0.0.19 documentation, 9월 30, 2025에 액세스, <https://analogdevicesinc.github.io/pyadi-iio/guides/examples.html>
 26. PLUTO HDL Project — HDL documentation - Repositories, 9월 30, 2025에 액세스, <https://analogdevicesinc.github.io/hdl/projects/pluto/index.html>
 27. Xilinx/PYNQ: Python Productivity for ZYNQ - GitHub, 9월 30, 2025에 액세스, <https://github.com/Xilinx/PYNQ>
 28. PYNQ SD Card image — Python productivity for Zynq (Pynq) - Read the Docs, 9월 30, 2025에 액세스, https://pynq.readthedocs.io/en/v3.1/pynq_sd_card.html
 29. PYNQ SD Card image — Python productivity for Zynq (Pynq) - Read the Docs, 9월 30, 2025에 액세스, https://pynq.readthedocs.io/en/latest/pynq_sd_card.html
 30. Quick Porting of PYNQ using Pre-built Images - Learn, 9월 30, 2025에 액세스, <https://discuss.pynq.io/t/quick-porting-of-pynq-using-pre-built-images/1075>
 31. Python Productivity for Zynq - GitHub, 9월 30, 2025에 액세스, https://raw.githubusercontent.com/Xilinx/PYNQ_Workshop/master/01_PYNQ_Workshop_introduction.pdf
 32. PYNQ SD Card — Python productivity for Zynq (Pynq) v1.0 - Read the Docs, 9월 30, 2025에 액세스, https://pynq.readthedocs.io/en/v2.1/pynq_sd_card.html
 33. how to add ad9361 to petalinux2018.2 - Adaptive Support - AMD, 9월 30, 2025에

엑세스,

https://adaptivesupport.amd.com/s/question/0D52E00006hpROkSAM/how-to-add-ad9361-to-petalinux20182?language=en_US

34. Petalinux tools can't find iio.h during build - Q&A - Linux Software Drivers - EngineerZone, 9월 30, 2025에 액세스,
<https://ez.analog.com/linux-software-drivers/f/q-a/575065/petalinux-tools-can-t-find-iio-h-during-build>
35. use ad9361 in petalinux - Q&A - Linux Software Drivers - EngineerZone - Analog Devices, 9월 30, 2025에 액세스,
<https://ez.analog.com/linux-software-drivers/f/q-a/577714/use-ad9361-in-petalinux>
36. Quick Start — Analog Devices Hardware Python Interfaces 0.0.19 documentation, 9월 30, 2025에 액세스,
<https://analogdevicesinc.github.io/pyadi-iio/guides/quick.html>
37. Examples — Analog Devices Hardware Python Interfaces 0.0.3 documentation, 9월 30, 2025에 액세스,
<https://pyadi-iio.readthedocs.io/en/latest/guides/examples.html>
38. PYNQ-Z1 Reference Manual - Digilent, 9월 30, 2025에 액세스,
<https://digilent.com/reference/programmable-logic/pynq-z1/reference-manual>
39. PYNQ Edition! Building PYNQ Images - Hackster.io, 9월 30, 2025에 액세스,
<https://www.hackster.io/news/pynq-edition-building-pynq-images-4f0ec30e172f>
40. Xilinx/RFSoc-PYNQ: Python productivity for RFSoc platforms - GitHub, 9월 30, 2025에 액세스, <https://github.com/Xilinx/RFSoc-PYNQ>
41. black-pigeon/antsdr-fw - GitHub, 9월 30, 2025에 액세스,
<https://github.com/black-pigeon/antsdr-fw>
42. FMCOMMS2/3/4 HDL Project - Repositories, 9월 30, 2025에 액세스,
<https://analogdevicesinc.github.io/hdl/projects/fmcomms2/index.html>
43. analogdevicesinc/pyadi-iio: Python interfaces for ADI hardware with IIO drivers (aka peyote), 9월 30, 2025에 액세스,
<https://github.com/analogdevicesinc/pyadi-iio>